

Yet another counting problem

Given a string S whose length can be more than 10^9 .

Given an array Arr and a string P array showing the count of characters, for example, array $Arr = [1, 2]$ and $P = "ab"$ then $S = "abb"$.

Count the number of sequences modulo 10^9+7 such that every sequence contains all vowels at least once.

Function description

Complete the *countSequence* function. This function takes the following 3 parameters and returns the required answer:

- N : Represents the size of the string P
- Arr : Represents values array of arr
- P : Represent the characters in the string P

Input format for custom testing

Note: Use this input format if you are testing against custom input or writing code in a language where we don't provide boilerplate code.

- The first line contains a single integer N .
- The next line contains N integer of array arr .
- The next line contains the string P .

Output format

```
5 int mod = 1e9+7;
6
7 ll power(long long x, unsigned int y, int p)
8 {
9     ll res = 1;
10
11     x = x % p;
12
13     if (x == 0) return 0;
14
15     while (y > 0)
16     {
17         if (y & 1)
18             res = (res*x) % p;
19
20         y >>= 1;
21         x = (x*x) % p;
22     }
23     return res;
24 }
```

```
26
27 int solve(int n, vector<int> &arr, string P){
28     string s = P;
29     vector<int> cnt(26, 0);
30     ll sum=0;
31     for(int i=0 ; i<n ; i++){
32         if(s[i] == 'a' || s[i] == 'e' || s[i] == 'i' || s[i] == 'o' || s[i] == 'u'){
33             cnt[s[i]-'a'] = arr[i];
34         }
35         else sum += arr[i];
36     }
37
38
39     if(cnt[0] == 0 || cnt['e'-'a'] == 0 || cnt['i'-'a'] == 0 || cnt['o'-'a'] == 0 || cnt['u'
    - 'a'] == 0){
40         return 0;
41     }
42
43     ll ans = 0;
44     ans = ((ll)ans+power(2, cnt[0], mod)-1)%mod;
45     ans = ((ll)ans*power(2, cnt[4], mod)-1)%mod;
46     ans = ((ll)ans*power(2, cnt['i'-'a'], mod)-1)%mod;
47     ans = ((ll)ans*power(2, cnt['o'-'a'], mod)-1)%mod;
48     ans = ((ll)ans*power(2, cnt['u'-'a'], mod)-1)%mod;
49     ans = ((ll)ans*power(2, sum, mod))%mod;
50
51     return ans;
52 }
53
```

Output format

Print the answer.

Constraints

$$1 \leq N \leq 100000$$

$$1 \leq Arr[i] \leq 100000$$

P contains lowercase English alphabets

Sample input

```
7
1 1 1 1 1 1 1
aeioubc
```

Sample output

```
4
```

Explanation

The valid sequence are "aeiou", "aeioub", "aeiouc" and "aeioubc".

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

1 limit

[Previous Question](#)

[Next Question](#)

```
1 > #include<bits/stdc++.h>
   int countSequence (int N, vector<int> arr, string P) {
3     // Write your code here
5     return 0;
6 > }
```

Chain square

Given a directed tree of N nodes in a form of an array A of size $N - 1$, such that there is a directed edge from the node $A[i]$ to the node $i+1$ ($1 \leq i < N$).

You can perform the following operations on it:

- In one operation, you can break any edge in the tree, dividing that tree into two trees.

Break the given tree into a set of trees such that each tree is a directed chain. Your score is the sum of squares of lengths of the directed chains in the set. Print the maximum possible score.

Notes

- A *directed chain* is a directed tree such that each node in that tree has at most one out-degree and *at most one in-degree*.
- The length of a directed chain is the number of nodes in that directed chain.
- Assume 1-based indexing.

Function description

Complete the `Max_Score()` function. This function takes the following 2 parameters and returns an integer (The maximum possible score):

- N : Represents the size of the tree

[Previous Question](#)

[Next Question](#)

```
4 long long dfs(int src, vector<vector<int>> &adj, long long &ans){
5
6     long long sum = 0, maxi=0;
7     for(int ad: adj[src]){
8         long long cur = dfs(ad, adj, ans);
9         ans += (long long) cur*cur;
10        maxi = max(maxi, cur);
11
12    }
13    ans -= (long long) maxi*maxi;
14    return maxi+1;
15 }
16
17 long long Max_Score(int N, vector<int> A){
18     vector<int> ar = A;
19     int n = N;
20
21     vector<vector<int>> adj(n+5);
22     for(int i=0 ; i<n-1 ; i++){
23         adj[ar[i]].push_back(i+2);
24     }
25
26     long long ans=0;
27     long long cur = dfs(1, adj, ans);
28     ans += (long long) cur*cur;
29     return ans;
30 }
```

Complete the `Max_Score()` function. This function takes the following 2 parameters and returns an integer (The maximum possible score):

- N : Represents the size of the tree
- A : Represents the $N-1$ elements of array A

Input format for custom testing

Note: Use this input format if you are testing against custom input or writing code in a language where we don't provide boilerplate code.

- The first line contains a single integer T , which denotes the number of test cases.
- For each test case:
 - The first line contains an integer N .
 - The second line contains $N-1$ space-separated integers denoting elements of the array A .

Output format

For each test case in a new line, print an integer representing the maximum possible score.

Constraints

$$1 \leq T \leq 10$$

$$1 \leq N \leq 10^5$$

$$1 \leq A_i \leq N$$

Sample input



Sample output

[Previous Question](#)



[Next Question](#)

Sample input

```

2
5
1 2 3 3
7
1 1 2 2 3 3

```

Sample output

```

17
15

```

Explanation

Here $T = 2$

Assumptions

- $N = 5$
- $A = [1, 2, 3, 3]$

Approach

- The edges are as follows: $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 4$, $3 \rightarrow 5$.
- Break the edge between node 3 and node 5.
- You end up with two directed chains of lengths 1 and 4.
- Score is $1 \cdot 1 + 4 \cdot 4 = 17$.

Assumptions

- $N = 7$
- $A = [1, 1, 2, 2, 3, 3]$

Approach

- The edges are as follows: $1 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 4$, $2 \rightarrow 5$, $3 \rightarrow 6$, $3 \rightarrow 7$.
- Break the edge between node 1 and node 2.

[Previous Question](#)

[Next Question](#)

Assumptions

- $N = 5$
- $A = [1, 2, 3, 3]$

Approach

- The edges are as follows: $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 4$, $3 \rightarrow 5$.
- Break the edge between node 3 and node 5.
- You end up with two directed chains of lengths 1 and 4.
- Score is $1*1 + 4*4 = 17$.

Assumptions

- $N = 7$
- $A = [1, 1, 2, 2, 3, 3]$

Approach

- The edges are as follows: $1 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 4$, $2 \rightarrow 5$, $3 \rightarrow 6$, $3 \rightarrow 7$.
- Break the edge between node 1 and node 2
- Break the edge between node 2 and node 4
- Break the edge between node 3 and node 6
- You end up with four directed chains of lengths 1, 1, 2, and 3.
- Score is $1*1 + 1*1 + 2*2 + 3*3 = 15$.

Note:

Your code must be able to print the sample output from the provided sample input. However, your code is run against multiple hidden test cases. Therefore, your code must pass these hidden test cases to solve the problem statement.

Limits

Time Limit: 1.0 sec(s) for each input file
Memory Limit: 256 MB

[Previous Question](#)

[Next Question](#)

```
1 > #include<bits/stdc++.h>
   long long Max_Score (int N, vector<int> A) {
   return 1;
5 > }
```